CSA4704-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 16: LSTM and RNN Performance Comparison

Problem Statement: A food-delivery platform wants to forecast daily order volume for the next week.

Long-term weekly and seasonal patterns matter. Two years of daily order counts with weekly and festival-season patterns (or an airline passenger dataset). The objective is to compare SimpleRNN and LSTM on a long-dependency forecasting task. Use TensorFlow, Keras, NumPy, Pandas, Matplotlib.

Tasks:

1. Prepare sequences with a 30-day look-back window.
2. Train a SimpleRNN and an LSTM with the same number of units.
3. Compare training loss, validation loss and training time.
4. Compute MAE, RMSE and MAPE on the test set.
5. Plot both forecasts against actual values; tabulate the results with Pandas.

FORMULA:

1. Sequence Preparation (30-day look-back window)

$$X_t = [x_{t-30}, x_{t-29}, \ldots, x_{t-1}]$$

$$Y_t = x_t$$

2. SimpleRNN Formula

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h)$$

$$\hat{y}_t = W_y h_t + b_y$$

3. LSTM Formula

Forget Gate:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Input Gate:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Candidate Cell State:

$$\widetilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

Cell State:

$$C_t = f_t \odot C_{t-1} + i_t \odot \widetilde{C}_t$$

Output Gate:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Hidden State:

$$h_t = o_t \odot \tanh(C_t)$$

4. Mean Squared Error (MSE)

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

5. Mean Absolute Error (MAE)

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

6. Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

7. Mean Absolute Percentage Error (MAPE)

$$MAPE = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$$

8. Training Time

$$T_{training} = T_{end} - T_{start}$$

Where:

- y_i = Actual order volume
- $\hat{y}_i$ = Predicted order volume
- n = Number of test samples
- h_t = Hidden state at time t
- C_t = LSTM cell state
- σ = Sigmoid activation function
- $\odot$ = Element-wise multiplication

CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
import time

np.random.seed(42)
tf.random.set_seed(42)

# Generate two years of daily order counts
days = np.arange(730)
orders = (200 + 30 * np.sin(2 * np.pi * days / 7)
        + 20 * np.sin(2 * np.pi * days / 365)
        + np.random.normal(0, 8, 730))
```

```
data = (orders - orders.mean()) / orders.std()

# Prepare 30-day sequences
X, y = [], []
for i in range(30, len(data)):
    X.append(data[i-30:i])
    y.append(data[i])

X = np.array(X)[..., np.newaxis]
y = np.array(y)

# Split data chronologically
a, b = int(len(X)*0.7), int(len(X)*0.85)
X_train, y_train = X[:a], y[:a]
X_val, y_val = X[a:b], y[a:b]
X_test, y_test = X[b:], y[b:]

results = []
forecasts = {}

# Train SimpleRNN and LSTM
for name in ["SimpleRNN", "LSTM"]:
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(30, 1)),
        (tf.keras.layers.SimpleRNN(32) if name == "SimpleRNN"
         else tf.keras.layers.LSTM(32)),
        tf.keras.layers.Dense(1)
    ])

    model.compile(optimizer="adam", loss="mse")

    start = time.time()
    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=15, batch_size=32, verbose=0
    )
    train_time = time.time() - start
```

```
    pred = model.predict(X_test, verbose=0).flatten()
    actual = y_test

    mae = np.mean(np.abs(actual - pred))
    rmse = np.sqrt(np.mean((actual - pred)**2))
    mape = np.mean(np.abs((actual - pred) /
                          np.maximum(np.abs(actual), 1e-8))) * 100

    results.append([
        name, history.history["loss"][-1],
        history.history["val_loss"][-1],
        train_time, mae, rmse, mape
    ])

    forecasts[name] = pred

# Display comparison table
columns = ["Model", "Train Loss", "Validation Loss",
           "Training Time (s)", "MAE", "RMSE", "MAPE (%)"]

print(pd.DataFrame(results, columns=columns).round(4))

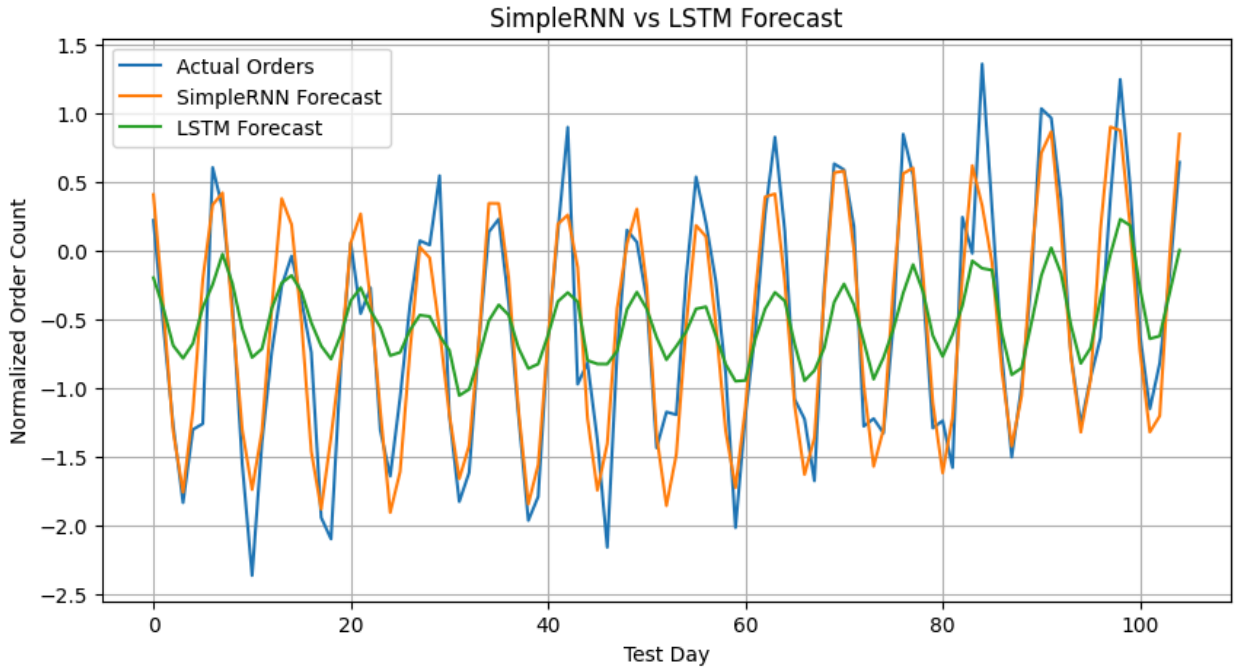
# Plot actual vs predicted orders
plt.figure(figsize=(10, 5))
plt.plot(actual, label="Actual Orders")
plt.plot(forecasts["SimpleRNN"], label="SimpleRNN Forecast")
plt.plot(forecasts["LSTM"], label="LSTM Forecast")
plt.xlabel("Test Day")
plt.ylabel("Normalized Order Count")
plt.title("SimpleRNN vs LSTM Forecast")
plt.legend()
plt.grid(True)
plt.show()

# Plot training and validation loss
for name, result in zip(["SimpleRNN", "LSTM"], results):
    print(name, "Training time:", round(result[3], 2), "seconds")
```

OUTPUT:

WARNING:tensorflow:5 out of the last 60 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7e4821266fc0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.
WARNING:tensorflow:6 out of the last 63 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7e4821266fc0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

	Model	Train Loss	Validation Loss	Training Time (s)	MAE	RMSE	MAPE (%)
0	SimpleRNN	0.1078	0.1146	16.4137	0.2719	0.3661	87.9825
1	LSTM	0.3084	0.4010	5.6717	0.5422	0.6575	122.4654



SimpleRNN Training time: 16.41 seconds
LSTM Training time: 5.67 seconds